

24MCADMT201 ADVANCED DATABASE MANAGEMENT SYSTEMS

UNIT 5:

System Architecture, Object Oriented Databases, XML and NoSQL:- Distributed Databases: Homogeneous and Heterogeneous Databases, Distributed Data Storage, Distributed Transactions - Object Based Databases: Overview, Complex Data types, Structured types and inheritance in SQL, Table Inheritance, Array and Multiset types in SQL, Object identity and reference types in SQL - XML: DTD and XML Schema, XML presentation, XML Applications - Next Generation Databases: Distributed Relational Databases - Nonrelational Distributed Databases - MongoDB Sharding and Replication - Hbase - Cassandra - CAP Theorem

System Architecture, Object Oriented Databases, XML and NoSQL

System Architecture

Advanced Database Management System (ADBMS), the system architecture typically involves several key components and layers. Here's an overview of the architecture:

User Layer

- **End Users:** Individuals who interact with the database through applications or interfaces.
- **Application Programs:** Software that communicates with the database, which can include web applications, mobile apps, and desktop applications.

Application Layer

- **Middleware:** Software that connects the user interface and the database, facilitating communication and data exchange.
- **Transaction Management:** Ensures database transactions are processed reliably and securely.

Database Layer

- **Database Management System (DBMS):** Core software that manages data storage, retrieval, and updating.
 - **Data Dictionary:** Contains metadata about the database structure, constraints, and relationships.
 - **Query Processor:** Translates queries into operations that the database can execute.
 - **Storage Manager:** Manages how data is stored, indexed, and retrieved from physical storage.

Storage Layer

- **Physical Storage:** Actual hardware where data is stored (e.g., hard drives, SSDs).
- **File Systems:** The way data files are organized and accessed on the storage hardware.

Network Layer

- **Communication Protocols:** Define how data is transmitted over networks, facilitating remote access to the database.
- **Database Servers:** Handle requests from clients and manage connections to the database.

Security Layer

- **Authentication and Authorization:** Mechanisms to ensure that only authorized users can access or modify data.
- **Encryption:** Protects data at rest and in transit to prevent unauthorized access.

Backup and Recovery Layer

- **Backup Systems:** Tools and processes for creating copies of data to prevent loss.
- **Recovery Management:** Mechanisms to restore data in case of failure or corruption.

Object Oriented Databases

Object-oriented databases (OODBs) integrate object-oriented programming principles with database management, allowing for more complex data modeling. Here's a detailed look at object-oriented databases and their architecture:

Key Concepts of Object-Oriented Databases

1. **Objects:** The fundamental units of data that encapsulate both state (attributes) and behavior (methods or functions).
2. **Classes:** Templates that define the structure of objects, including attributes and methods.
3. **Inheritance:** A mechanism to create new classes based on existing ones, promoting code reuse and establishing relationships.
4. **Encapsulation:** The bundling of data and methods that manipulate that data within objects, allowing for data hiding.
5. **Polymorphism:** The ability to process objects differently based on their data type or class.

Architecture of Object-Oriented Databases

The architecture of object-oriented databases can be outlined in several layers:

User Layer

- **End Users:** Individuals who interact with the database via applications.
- **User Interfaces:** Graphical or command-line interfaces that allow users to query and manage data.

Application Layer

- **Application Programs:** Software applications that utilize the database. These programs can perform operations like creating, reading, updating, and deleting objects.
- **Object Query Language (OQL):** A specialized query language designed for object-oriented databases, analogous to SQL but tailored for object retrieval.

Object Layer

- **Object Database Management System (ODBMS):** The core of the architecture that manages the storage and retrieval of objects.
 - **Object Store:** A repository for storing objects, which can include various types of storage (e.g., files, memory).
 - **Object Manager:** Handles the lifecycle of objects, including creation, deletion, and modification.
 - **Persistence Manager:** Ensures that objects can be stored and retrieved efficiently from the object store.

Storage Layer

- **Physical Storage:** The underlying hardware where object data is stored. This may involve specialized storage systems optimized for object retrieval.
- **File Organization:** Structures used to store objects on disk, often implemented with techniques such as object identity and reference mapping.

Network Layer

- **Communication Protocols:** Define how data is transmitted between the client applications and the ODBMS, facilitating remote access.
- **Database Servers:** Handle requests from client applications and manage connections to the database.

Security Layer

- **Authentication and Authorization:** Mechanisms to ensure that users have appropriate permissions to access or modify objects.
- **Data Encryption:** Protects sensitive data stored in the database from unauthorized access.

Backup and Recovery Layer

- **Backup Mechanisms:** Procedures for creating backup copies of object data to prevent loss.
- **Recovery Tools:** Systems in place to restore data in case of failure or corruption.

Advantages of Object-Oriented Databases

- **Complex Data Representation:** OODBs can naturally model complex data relationships and structures.
- **Code Reusability:** Inheritance allows developers to reuse existing code, making development more efficient.
- **Data Integrity:** Encapsulation ensures that data is accessed and modified only through defined methods, enhancing data integrity.
- **Better Performance:** Optimized for managing complex data types and relationships, potentially improving performance in certain applications.

Applications of Object-Oriented Databases

- **Multimedia Databases:** Storing and managing complex data types like images, audio, and video.
- **CAD/CAM Systems:** Used in engineering applications to manage complex designs.
- **Geographic Information Systems (GIS):** Handling spatial data and relationships.

XML and NoSQL: Distributed Databases

XML (eXtensible Markup Language) and **NoSQL** databases are two distinct technologies that cater to different data storage and management needs. Both can be used in distributed database systems, but they serve different purposes.

XML (eXtensible Markup Language)

- **Definition:** XML is a markup language that defines rules for encoding documents in a format that is both human-readable and machine-readable.
- **Structure:** XML organizes data in a hierarchical, tree-like structure using tags, which makes it suitable for representing complex data relationships.

Key Features

- **Flexibility:** XML allows users to define their own tags, making it highly adaptable for various applications.
- **Interoperability:** It is platform-independent and can be used across different systems and applications.
- **Self-descriptive:** XML documents contain metadata about the structure and meaning of the data.

Use Cases

- **Data Interchange:** Commonly used for data exchange between systems, especially in web services (e.g., SOAP).
- **Configuration Files:** Used to store configuration settings in many applications.
- **Content Management:** Helps in managing documents and content in a structured way.

Distributed XML Databases

- **Storage:** XML data can be stored in distributed databases that support XML natively, allowing for efficient querying and retrieval.
- **Querying:** Technologies like XQuery and XPath are used to query XML data, even in distributed environments.

NoSQL Databases

- **Definition:** NoSQL databases are a class of databases designed to handle large volumes of unstructured or semi-structured data. They provide flexible schemas and are often designed for horizontal scaling.
- **Types:** Common types include document stores, key-value stores, column-family stores, and graph databases.

Key Features

- **Scalability:** Designed to scale out by distributing data across many servers, making them suitable for big data applications.
- **Flexible Schema:** Unlike relational databases, NoSQL databases allow for dynamic schemas, enabling easy changes to the data model.
- **High Performance:** Optimized for read and write operations, making them suitable for high-throughput applications.

Use Cases

- **Big Data Applications:** Handling large datasets, such as social media data, IoT data, and logs.
- **Real-time Analytics:** Suitable for applications requiring quick data retrieval and processing.
- **Content Management:** Storing content in formats that do not fit well into traditional relational databases.

Distributed NoSQL Databases

- **Architecture:** NoSQL databases are often designed to be distributed from the ground up, allowing for data to be spread across multiple nodes.
- **Data Replication:** Data can be replicated across different nodes for fault tolerance and availability.
- **Partitioning:** Data is often partitioned (sharded) to distribute load and improve access times.

Comparison: XML vs. NoSQL in Distributed Databases

Feature	XML	NoSQL
Data Structure	Hierarchical, tree-like	Various (document, key-value, etc.)
Schema	Flexible but fixed at document level	Highly flexible, dynamic
Scalability	Limited; typically not horizontally scalable	Designed for horizontal scalability
Query Language	XQuery, XPath	Varies by database (e.g., MongoDB query language, CQL)
Use Cases	Data interchange, document management	Big data, real-time analytics

	Relational Databases	NoSQL Databases
Data Structure	Structured, tabular schema, fixed data types.	Schema-less accommodates a variety of data types.
Scalability	Typically scaled vertically (more CPU, RAM, SSD).	Designed for horizontal scaling (more servers).
Flexibility	Limited flexibility.	High flexibility.
ACID Properties	Adhere to ACID properties for reliable transactions.	Some sacrifice ACID compliance for performance and scalability, others offer tunable consistency.
Query Language	Uses SQL for defining and manipulating data.	Often uses simpler query languages, may provide APIs for data manipulation.
Relations	Can handle relations between different data points effectively.	Some can handle relations effectively (like graph databases), but not all are designed to do so.
Use Cases	Ideal for applications requiring complex queries, and multi-row transactions: financial systems, e-commerce applications.	Used for large data sets, real-time applications, and high write volume systems: social media, big data applications.

Homogeneous Databases

Homogeneous databases are characterized by all participating nodes sharing the same database management system (DBMS) and schema structure. These databases are designed to offer a unified and consistent view of data across all nodes.

Characteristics of Homogeneous Databases

- **Uniformity:** All nodes utilize the same DBMS software and possess identical database schemas.
- **Data Consistency:** Changes made to the database on one node are automatically propagated to other nodes, ensuring data consistency.
- **Simplicity:** Homogeneous databases are relatively easier to manage as the same DBMS software is employed throughout the system.

Steps to Set up a Homogeneous Database:

- **Select a DBMS:** Choose a DBMS that aligns with the distributed system's requirements, such as MySQL.
- **Install the DBMS:** Install the chosen DBMS on each node within the distributed system.
- **Design the Schema:** Create a unified database schema to be shared across all nodes.
- **Establish Communication:** Configure network connectivity between the nodes to facilitate data replication and synchronization.
- **Implement Replication:** Set up replication mechanisms provided by the DBMS to ensure changes made on one node are propagated to others.

Example:

Imagine a company with multiple branches, each using MySQL as their database system. Each branch has its own MySQL server, but they all follow the same schema and can communicate with each other seamlessly.

Advantages:

Easier to manage and maintain due to uniformity.
Simplified data integration and querying.

Disadvantages:

Limited flexibility in choosing different DBMS for different needs.

Heterogeneous Databases

Contrary to homogeneous databases, heterogeneous databases allow nodes in a distributed system to use different DBMS software or possess varying database schemas. This approach caters to diverse requirements and facilitates seamless integration between nodes employing different technologies.

Characteristics of Heterogeneous Databases

- **Flexibility:** Nodes can employ different DBMS software, such as MySQL, MongoDB, or Cassandra, based on their specific needs.
- **Schema Mapping:** Heterogeneous databases necessitate mapping between different schemas to ensure interoperability between nodes.

- **Data Transformation:** Data might need to be transformed or translated between different formats or encodings to maintain consistency.

Steps to Set up a Heterogeneous Database

- **Identify Diverse Requirements:** Understand the specific needs of each node in the distributed system and select the appropriate DBMS software accordingly.
- **Define Schema Mapping:** Analyze the differences in database schemas between nodes and establish mapping rules to convert data between schemas.
- **Implement Data Transformation:** Develop mechanisms or scripts to transform data from one format to another, ensuring seamless integration.
- **Establish Communication:** Configure network connectivity and establish communication channels between heterogeneous nodes.

Example:

Consider a scenario where a retail company uses **Oracle** for its financial data, **MongoDB** for its customer data, and **PostgreSQL** for its inventory data. Each system operates independently, but they are part of the same distributed architecture.

Explanation:

- **Advantages:**
 - Flexibility to choose the best DBMS for specific needs.
 - Can leverage the strengths of different systems.
- **Disadvantages:**
 - Increased complexity in data integration and querying.
 - Requires more effort to manage and maintain.

Distributed Data Storage

Definition: Distributed data storage refers to a method of storing data across multiple physical locations, which can be on different servers or nodes in a network. This approach enhances data availability, reliability, and scalability.

Example:

Imagine a company that has offices in different countries. Instead of storing all data in a single server, the company uses distributed storage to keep data close to each office.

- **Office A** in the USA stores customer data.
- **Office B** in Europe stores inventory data.
- **Office C** in Asia stores sales data.

When an employee in Office A needs customer information, they access the data from their local server, which reduces latency and improves performance.

Advantages of Distributed Storage Systems

- **Scalability:** By adding more storage nodes or servers, distributed storage systems enable enterprises to grow in response to increasing data volumes and user needs.
- **Fault Tolerance:** Distributed storage systems can withstand hardware failures by replicating data across several nodes, guaranteeing data availability and uninterrupted service.
- **High Availability:** In the event of hardware malfunctions or network outages, redundancy and fault tolerance techniques guarantee continuous access to data and services.

- **Performance:** By dividing up the workload and data among several nodes, distributed storage systems can lower latency and bottlenecks and increase performance.
- **Cost-Effectiveness:** Distributed storage systems can offer more affordable storage solutions than conventional monolithic storage systems by utilizing scalable architectures and commodity technology.

Dis-advantages of Distributed Storage Systems

- **Complexity:** Distributed systems, networking, and data management expertise are required for the design, deployment, and maintenance of distributed storage systems, which can be challenging.
- **Consistency Issues:** It can be difficult to maintain data coherence and consistency among dispersed nodes, particularly in settings with a lot of concurrency and rapid changes.
- **Network Overhead:** Performance and bandwidth utilization may be impacted by network overhead caused by replicating data among several nodes and coordinating updates.
- **Security Concerns:** Robust security measures and access controls are necessary because distributed storage systems may pose extra security risks, such as illegal access, data breaches, and compliance concerns.

Distributed Transactions

Definition: A distributed transaction is a transaction that involves multiple databases or data sources. It ensures that all parts of the transaction are completed successfully or none at all (atomicity).

Example:

Consider an online shopping platform where a customer wants to buy a product that requires a payment and an update to the inventory.

1. Transaction Steps:

- **Step 1:** Deduct the payment from the customer's bank account (Bank Database).
- **Step 2:** Update the inventory to reflect the new stock level (Inventory Database).

2. **Atomicity:** If the payment is deducted but the inventory update fails (e.g., due to a network issue), the entire transaction must be rolled back. The payment will be reversed, ensuring the customer's data remains consistent.

Implementing Distributed Transactions

• Transaction Managers (TM):

- Transaction Managers are responsible for coordinating and managing transactions across multiple resource managers (e.g., databases, message queues).
- TMs ensure that transactions adhere to ACID properties (Atomicity, Consistency, Isolation, Durability) even when involving disparate resources.

• Resource Managers (RM):

- Resource Managers are responsible for managing individual resources (e.g., databases, file systems) involved in a distributed transaction.
- RMs interact with the TM to prepare for committing or rolling back transactions based on the TM's coordination.

• Coordination Protocols:

- Implementations of distributed transactions often rely on coordination protocols like 2PC, 3PC, or variants such as Paxos and Raft for consensus.

- These protocols ensure that all participants in a transaction reach a consistent decision regarding commit or rollback.

Advantages of Distributed Transactions

- **Data Consistency:** Data Consistency is being provided across multiple resources by distributed transactions. Various Operations are being coordinated across multiple database resources. This makes sure that system remains in a consistent state even in case of any type of failure.
- **Fault Tolerance:** Distributed systems can handle faults and ensure proper transactions. If the participating resource fails during the execution of the transaction the transaction can be then rolled back on alternate resources and completed successfully.
- **Guarantees Transactions:** Distributed systems guarantee the transaction. It provides features such as durability and isolation. The durability makes sure that if any transaction is committed, the changes last even if any failures occur.

Applications Distributed Transactions

- **Enterprise Resource Planning (ERP) Systems:** ERP systems consist of departments within one organization. Therefore distributed transactions are used here in order to maintain transactions from various modules such as sales, inventory, finance, and human resources management.
- **Cloud Computing:** Distributed transactions are being used in cloud-based applications. Transactions can be done with the help of multiple data sources and ensure that data updates and operations that are performed consistently.
- **Healthcare Systems:** Healthcare systems make use of Distributed transactions when coordinating patient records, scheduling appointments for patients, and managing the billing systems. Distributed transactions maintain data consistency and performance in healthcare systems.

Object Based Databases: Overview

Object-based databases integrate object-oriented programming concepts into database systems. They are designed to handle complex data types and relationships more effectively than traditional relational databases. Here's an overview of some key concepts:

Overview

Object-based databases allow for the storage of objects directly in the database. Unlike relational databases, where data is stored in tables, object databases store data as objects, similar to how it is represented in programming languages. This is useful for applications that require the storage of complex data structures.

Complex Data Types

Object databases support complex data types, which can include:

- **User-defined Types (UDTs):** Custom data types defined by the user.
- **Multisets:** Collections of elements that can include duplicates.
- **Arrays:** Ordered collections of elements.

Advantages of Complex Data Types

- **Rich Data Modeling:** Complex data types allow for more accurate representations of real-world entities and relationships.
- **Encapsulation:** Related attributes can be encapsulated within a single type, improving organization.

- **Flexibility:** They enable developers to create customized data structures that suit specific application needs.
- **Simplified Queries:** Complex types can lead to simpler and more readable queries when dealing with structured data.

Structured Types and Inheritance in SQL

Structured types are a specific kind of UDT(User Defined Type) that groups related fields together. They can represent real-world entities more naturally, allowing for complex attributes.

```
CREATE TYPE Person AS (
    name VARCHAR(50),
    age INT,
    address Address -- Using the Address UDT
);
```

Here, the Person type includes a field of type Address, demonstrating how structured types can encapsulate other complex types.

Inheritance allows a new type to inherit attributes and methods from another type. This promotes reusability and simplifies the structure.

Example of Inheritance in SQL

```
CREATE TYPE Person AS (
    name VARCHAR(50),
    age INT
);
```

```
CREATE TYPE Employee UNDER Person AS (
    employee_id INT,
    department VARCHAR(50)
);
```

Table Inheritance

In SQL, table inheritance allows a table to inherit the structure of another table. This is particularly useful for representing hierarchical data.

Example of Table Inheritance

```
CREATE TABLE Person (
    person_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

```
CREATE TABLE Employee (
    employee_id SERIAL PRIMARY KEY,
```

```
department VARCHAR(50)  
) INHERITS (Person);
```

In this example, the Employee table inherits the name and age columns from the Person table.

Array and Multiset Types in SQL

Array Types allow storing multiple values in a single column.

Example of Array in SQL

```
CREATE TABLE Courses (  
    student_id INT,  
    course_names VARCHAR(100[])  
);
```

This example creates a table where course_names can store an array of course names.

Multiset Types allow for storing collections of items that may have duplicates.

Example of Multiset in SQL

```
CREATE TABLE ShoppingCart (  
    user_id INT,  
    items MULTiset<ITEM_TYPE>  
);
```

Object Identity and Reference Types in SQL

Object Identity refers to the unique identification of objects. Each object has a unique identifier (OID) that distinguishes it from other objects.

Reference Types allow one object to refer to another, creating relationships between objects.

Example of Object Identity and Reference Types

```
CREATE TYPE Product AS (  
    product_id SERIAL PRIMARY KEY,  
    name VARCHAR(50),  
    price DECIMAL  
);
```

```
CREATE TYPE Order AS (  
    order_id SERIAL PRIMARY KEY,  
    product_ref REF Product  
);
```

In this example, the Order type contains a reference to the Product type, allowing it to link to specific products.

XML: DTD and XML Schema, XML presentation, XML Applications

Document Type Definition and XML Schema Definition both contain a set of rules that control the structure and elements of XML files, but they are different in syntax and usage. They check whether an XML document has a valid structure or not.

Document Type Definition (DTD)

DTD stands for Document Type Definition and DTD contains a set of rules that control the structure and elements of XML files. It is used to describe the attributes of XML language precisely. It can be classified into two types namely internal DTD and external DTD. It can be specified inside a document or outside a document. DTD mainly checks the grammar and validity of an XML document.

Example of an Internal DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE book [
  <!ELEMENT book (title, author, publisher, price)>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT publisher (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
]>
<book>
  <title>Advanced Databases</title>
  <author>John Doe</author>
  <publisher>Tech Books</publisher>
  <price>50</price>
</book>
```

DTD Explanation:

- <!ELEMENT>: Defines the structure of elements.
- (#PCDATA): Defines that the element contains parsed character data (essentially text).
- <!DOCTYPE>: Declares the DTD for the XML document.

Features of DTD

- It defines the compulsory and optional elements in the XML document.
- It validates the structure of the XML document.
- It checks for the grammar of the XML document.
- It describes the order in which the element occurs.

Advantages of DTD

- We can define our own format for the XML files by DTD.
- It helps in validation of XML file.

- It provides us with a proper documentation.
- It enables us to describe a XML document efficiently.

Disadvantages of DTD

- DTDs are hard to read and maintain if they are large in size.
- It is not object oriented.
- The documentation support is limited.
- DTD doesn't support namespaces.

XML Schema Definition (XSD)

An **XML Schema** (often called **XSD** for XML Schema Definition) is a more advanced and powerful way to define the structure and data types of XML documents. It is a W3C standard and provides greater flexibility than DTDs.

XML Schema defines:

- **Element types** (e.g., string, integer, date)
- **Element order**
- **Data constraints** (e.g., a number must be positive, or a string must have a minimum length)
- **Complex types** (e.g., lists or nested elements)

Example of an XML Schema (XSD):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="publisher" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Features of XSD

- XSD allows you to create detailed data arrangements with things like nested elements, attributes, and different data types.
- Helps you define different kinds of data, like words, numbers, and dates, in XML documents. It allows you to decide how this data should be structured.
- You can also decide on default values and rules for elements and attributes in your XML documents.

- XSD is good for working with various names in XML, which is useful when you're creating rules for complex XML files with lots of different names.

Advantages of XSD

- XSD schemas are like user-friendly guides for XML data. They tell developers how the data should be structured and the rules it must follow, all without needing extra explanations.
- Helps different systems and apps work together smoothly by creating a common language for organizing and checking data.
- Lets you set automatic values for missing parts in your data. It's like having placeholders ready, so if some information is missing, it gets filled in with the default values. This keeps your data consistent.
- XSD checks data to make sure it's correct, stopping any mistakes from getting into XML documents.
- Provides different data types, like primitive data types, derived data types, and user-defined data types. This variety helps keep data organized and dependable.

Disadvantages of XSD

- Creating and maintaining XSD schemas can be complex, especially for intricate XML structures.
- Understanding and working with XSD can be challenging for those new to XML and related technologies.
- Data validation using XSD can introduce a performance overhead, especially for large XML documents.

Example :

Basic Structure of an XML Document:

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
  <title>Advanced Databases</title>
  <author>John Doe</author>
  <publisher>Tech Books</publisher>
  <price>50</price>
</book>
```

XML Presentation

XML Presentation refers to how XML data is displayed or presented to users. While XML itself is primarily a data representation format, it doesn't focus on the visual presentation. However, we can use technologies like **XSLT** (Extensible Stylesheet Language Transformations) and **CSS** to transform and present XML data in a human-readable or structured format.

- **XSLT**: A language for transforming XML documents into other formats (like HTML, plain text, or other XML documents).

- **CSS:** Used for styling XML documents, particularly when transforming XML into HTML for display in browsers.

Example of an XSLT Transformation:

Here's an XML file representing a list of books:

```
<books>
  <book>
    <title>Advanced Databases</title>
    <author>John Doe</author>
    <publisher>Tech Books</publisher>
    <price>50</price>
  </book>
  <book>
    <title>Intro to Databases</title>
    <author>Jane Smith</author>
    <publisher>Tech Books</publisher>
    <price>40</price>
  </book>
</books>
```

And an XSLT to transform it into HTML:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/books">
    <html>
      <body>
        <h2>Book List</h2>
        <table border="1">
          <tr>
            <th>Title</th>
            <th>Author</th>
            <th>Publisher</th>
            <th>Price</th>
          </tr>
          <xsl:for-each select="book">
            <tr>
              <td><xsl:value-of select="title"/></td>
              <td><xsl:value-of select="author"/></td>
              <td><xsl:value-of select="publisher"/></td>
              <td><xsl:value-of select="price"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </template>
</xsl:stylesheet>
```

```
</html>
</xsl:template>
</xsl:stylesheet>
```

When this XSLT is applied to the XML, it produces an HTML table for display.

XML Applications in ADBMS

- **Data Exchange:** XML is often used for data interchange between different systems and applications, providing a standard format.
- **Data Storage:** Some ADBMS support XML data types, allowing for the storage of XML documents directly in the database.
- **Querying XML:** Advanced querying mechanisms (like XQuery) allow for querying XML data stored in databases.
- **Integration:** XML facilitates integration between heterogeneous systems, enabling communication and data sharing.

Next Generation Databases: Distributed Relational Databases - Nonrelational Distributed Databases

Distributed Relational Databases (RDBMS)

Relational databases (RDBMS) use a structured format for data storage, typically in tables with rows and columns. The data is managed using Structured Query Language (SQL).

In a **distributed relational database**, the data is split across different nodes (servers or computers), but the structure still maintains the relational model. In this system, data is distributed but still follows the principles of tables, primary/foreign keys, and ACID (Atomicity, Consistency, Isolation, Durability) transactions.

Key Features:

- **Sharding:** The data is partitioned across multiple servers. Each partition (or shard) holds a subset of the data.
- **Replication:** Copies of the data can be stored across multiple servers to provide redundancy and ensure high availability.
- **Consistency Models:** Distributed relational databases usually follow strict consistency rules to ensure data integrity (often at the cost of availability).

Example:

Consider a simple **E-commerce Database** with two tables: Customers and Orders.

- **Customers Table:** Stores customer information like customer_id, name, email, etc.
- **Orders Table:** Stores orders information like order_id, customer_id, order_date, total_amount.

In a **distributed relational database** system, the Customers table could be stored on one server while the Orders table is stored on another server. These tables are still related through the customer_id field, and SQL queries can be executed to join these tables across servers.

Query example:

```
SELECT c.name, o.total_amount
FROM Customers c
```

JOIN Orders o ON c.customer_id = o.customer_id;

Distributed relational databases like **Google Spanner**, **Amazon Aurora**, and **CockroachDB** are examples of systems that provide relational database features in a distributed environment.

Non-relational Distributed Databases (NoSQL)

Non-relational (NoSQL) databases differ from relational databases in that they do not use tables with rows and columns. Instead, they store data in formats like documents, key-value pairs, graphs, or wide-column stores. They are often more flexible in how they store and manage data.

Non-relational distributed databases can handle a variety of data types, and they typically allow for easier horizontal scaling, meaning they can scale out more easily by adding more machines.

Key Features:

- **Schema-less:** Data does not require a predefined schema like relational databases.
- **Eventual Consistency:** Many NoSQL systems prioritize availability and partition tolerance over strict consistency, meaning that some data inconsistencies might exist temporarily.
- **High Availability and Scalability:** They are designed to handle large volumes of unstructured or semi-structured data with low latency.

Types of NoSQL Databases:

- **Document-based** (e.g., MongoDB, CouchDB): Stores data in documents (JSON, BSON).
- **Key-Value Stores** (e.g., Redis, DynamoDB): Stores data as key-value pairs.
- **Column-family Stores** (e.g., Cassandra, HBase): Stores data in columns rather than rows.
- **Graph Databases** (e.g., Neo4j, ArangoDB): Stores data as graphs with nodes and edges.

Example:

Let's take a **social media platform** with two collections: Users and Posts.

- **Users Collection:** Each document represents a user with fields like user_id, name, email, etc.
- **Posts Collection:** Each document represents a post with fields like post_id, user_id, content, timestamp, etc.

In a **distributed non-relational database** (like MongoDB), the Users and Posts data could be stored across multiple servers. MongoDB automatically handles distributing data across shards. To fetch a user's posts, we could run a query like:

```
db.posts.find({ "user_id": "user123" })
```

In NoSQL databases, queries are typically more flexible because the data doesn't need to conform to a strict schema.

Comparing Relational and Non-relational Distributed Databases

Feature	Distributed Relational Databases	Non-relational Distributed Databases
---------	----------------------------------	--------------------------------------

Data Model	Tables with rows and columns	Flexible models like key-value, documents, graphs, etc.
Schema	Fixed schema, requires predefined structure	Schema-less, flexible data structures
Scalability	Horizontal scalability is more complex	Better horizontal scalability
Consistency	Strong consistency (ACID transactions)	Eventual consistency or tunable consistency
Examples	Google Spanner, Amazon Aurora, CockroachDB	MongoDB, Cassandra, Redis, DynamoDB, Couchbase
Use Cases	Financial systems, enterprise applications, data integrity-focused apps	Real-time analytics, social media, content management

MongoDB Sharding and Replication

MongoDB Sharding allows you to distribute your data across multiple servers, providing horizontal scalability to handle large datasets. **MongoDB Replication** ensures high availability and data redundancy by maintaining copies of your data across multiple nodes. By combining **sharding** and **replication**, you can build a system that is highly scalable and highly available, able to handle massive amounts of data while minimizing downtime.

MongoDB Sharding

Sharding is the process of distributing data across multiple machines or clusters to ensure that the database can scale horizontally. Instead of having one single server handle all the data, **sharding** divides the data into smaller chunks, which are spread across multiple servers (called **shards**). Each shard holds a subset of the data, and together, they form the complete dataset.

Why Use Sharding?

- **Scalability:** When your dataset grows too large for a single server to handle, sharding helps distribute the load across multiple machines.
- **Performance:** By distributing data, sharding helps balance the workload and improve query performance.

How Does Sharding Work in MongoDB?

MongoDB uses a **shard key** to determine how to split the data into chunks. The **shard key** is a field in the document that MongoDB uses to distribute data across shards. The choice of a shard key is crucial for balanced data distribution and performance.

MongoDB shards data in the following way:

- **Shards:** The actual servers that store data.
- **Config Servers:** Manage metadata and configuration information about the sharded cluster.
- **Mongos:** Acts as the query router, directing client requests to the appropriate shard.

Sharding Example

Consider a collection of documents in the **orders** collection for an e-commerce platform. Each document might look like this:

```
{
  "_id": ObjectId("..."),
  "order_id": 12345,
  "customer_id": 67890,
```

```
"total_amount": 250.0,  
"order_date": "2025-03-19T14:30:00Z"  
}
```

To shard the **orders** collection, we must select a **shard key**. In this case, let's choose **customer_id** as the shard key. MongoDB will use the **customer_id** to determine how to distribute the documents across multiple shards.

Step 1: Choose the Shard Key:

- In this case, we will shard by **customer_id**.

Step 2: Create the Sharded Cluster (for example purposes, we will use 3 shards):

```
sh.enableSharding("ecommerce") # Enable sharding on the "ecommerce" database  
sh.shardCollection("ecommerce.orders", { "customer_id": 1 }) # Shard the "orders" collection  
by "customer_id"
```

MongoDB will automatically divide the data based on the customer_id. For example:

- **Shard 1** might hold orders where **customer_id** is between 1 and 1000.
- **Shard 2** might hold orders where **customer_id** is between 1001 and 2000.
- **Shard 3** might hold orders where **customer_id** is between 2001 and 3000.

When querying the orders collection, MongoDB uses the Mongos query router to direct requests to the correct shard based on the shard key.

Example query:

```
db.orders.find({ "customer_id": 1200 }) # This would go to the shard where customer_id <= 2000.
```

Sharding Benefits:

- It allows horizontal scaling by distributing the load across multiple servers.
- MongoDB can scale out as your data grows, handling massive datasets more efficiently.

Challenges in Sharding:

- Choosing the right **shard key** is crucial. A poor choice can lead to uneven distribution of data, causing some shards to be overloaded (hotspots).
- Complex queries that do not use the shard key may require querying all shards, leading to performance issues.

MongoDB Replication

Replication in MongoDB involves maintaining copies of data across multiple servers (or nodes). The goal is to provide **high availability** and **data redundancy**. If one node fails, another node can take over without data loss.

How Does Replication Work in MongoDB?

MongoDB uses a **Replica Set**, which consists of:

- **Primary Node:** The main node that handles all read and write operations.
- **Secondary Nodes:** Replicas of the primary node. They maintain copies of the data and can serve read requests.
- **Arbiter:** A special node that doesn't hold data but participates in elections to elect a primary node.

How Replication Works:

- The **Primary Node** receives all the writes.
- **Secondary Nodes** replicate the data from the primary node. The secondaries can be configured to allow read operations, which can help balance the load.
- If the primary node goes down, the replica set automatically elects a new primary to maintain availability.

Replication Example

Step 1: Set up a Replica Set: Let's say you want to set up a replica set with three nodes: primary, secondary1, and secondary2.

Start MongoDB on each node, and initiate the replica set:

```
rs.initiate({  
  _id: "rs0", # Replica set name  
  members: [  
    { _id: 0, host: "primary:27017" }, # Primary node  
    { _id: 1, host: "secondary1:27017" }, # Secondary node 1  
    { _id: 2, host: "secondary2:27017" } # Secondary node 2  
  ]  
})
```

Step 2: Insert data into the orders collection:

```
db.orders.insertOne({ order_id: 12345, customer_id: 67890, total_amount: 250.0 })
```

Step 3: MongoDB automatically replicates this data to the secondary nodes. The secondary nodes maintain a copy of the orders collection, ensuring data redundancy.

Step 4: Query data from a secondary node:

```
db.orders.find({ "customer_id": 67890 })
```

By default, queries will go to the primary node, but you can configure the secondary nodes to serve reads as well.

Replication Benefits:

- **High Availability:** If the primary node goes down, another secondary node is automatically promoted to primary.
- **Read Scaling:** Secondary nodes can serve read requests, balancing the read load.
- **Data Redundancy:** Data is replicated across multiple nodes, ensuring no data loss in case of hardware failure.

Challenges in Replication:

- **Replication Lag:** There might be a small delay in replicating data from the primary to the secondary nodes.
- **Write Availability:** While secondary nodes can serve read requests, all write operations must go through the primary node.

HBase

HBase is a distributed, scalable, and highly available **NoSQL** database that is designed to store and manage large volumes of structured data. It is particularly optimized for **real-time read/write access to big data**. HBase is part of the **Apache Hadoop ecosystem** and is modeled after Google's **Bigtable**. It is used primarily for applications that require **high throughput** and **low-latency** access to massive datasets.

Key Features of HBase

1. Distributed and Scalable:

- HBase is designed to scale out horizontally, meaning you can add more servers to your HBase cluster to increase storage and throughput.
- It is built on top of **HDFS (Hadoop Distributed File System)**, which provides fault tolerance and scalability.

2. NoSQL Database:

- HBase is a **NoSQL** database, which means it doesn't follow the traditional relational model with tables, rows, and columns.
- Instead, it stores data in a **wide-column store** format, making it ideal for applications that need to handle large volumes of unstructured or semi-structured data.

3. Real-Time Access:

- HBase allows for low-latency access to data. It is optimized for real-time read and write operations, especially in scenarios where quick access to large datasets is required.

4. Column-Oriented Storage:

- Data in HBase is organized into **tables** consisting of **rows and columns**. However, unlike traditional relational databases, HBase organizes data into **column families**. A column family is a set of columns that are grouped together, which can improve read performance by storing related data physically close to each other.

5. Automatic Sharding and Region Distribution:

- HBase automatically divides large tables into smaller chunks called **regions**. These regions are distributed across the cluster to balance the workload.
- The system can add more regions as needed, allowing for elastic scaling.

6. Fault Tolerance:

- HBase uses **HDFS** for storage, which provides **replication** and **fault tolerance**. If a node in the cluster fails, HBase can recover without data loss by replicating data across multiple servers.

7. Consistency:

- HBase is **eventually consistent** (like many NoSQL databases) meaning that updates to data may take time to propagate across all replicas. However, it guarantees strong consistency within a single row.

HBase Architecture

HBase follows a **master-slave architecture** with three main components:

1. Region Servers:

- These servers are responsible for handling read and write requests. Each region server manages a subset of the **regions** of a table. A **region** is essentially a slice of the data in the table.
- Region servers are the workhorses of the system, responsible for responding to client requests and managing region splits.

2. HBase Master:

- The **HBase Master** is responsible for managing the cluster, including:
 - Assigning regions to region servers.
 - Handling region splits and merges.
 - Monitoring the health of the cluster.

3. ZooKeeper:

- **ZooKeeper** is used to coordinate and manage the distributed aspects of HBase. It helps with the **location** of region servers, **failover handling**, and **configuration management**.

4. HDFS (Hadoop Distributed File System):

- Data in HBase is stored in **HDFS**. Each cell of a table in HBase corresponds to a **key-value pair** stored in HDFS.
- HDFS handles the physical storage of the data and ensures fault tolerance through **data replication**.

HBase Data Model

The HBase data model is inspired by Google's **Bigtable** and consists of the following components:

- **Tables:** A table consists of rows and columns, where each row is uniquely identified by a **row key**.
- **Column Families:** A table can have one or more column families. Each column family contains multiple columns.
- **Columns:** Columns belong to a specific column family and are grouped together in the same storage file.

Example:

Here's an example of how data might be represented in HBase:

- **Table:** users
 - **Row Key:** user_id
 - **Column Family 1:** personal_info
 - name
 - age
 - email
 - **Column Family 2:** address_info
 - street
 - city
 - zipcode

A row in HBase for user_id = 1001 might look like this:

Row Key	Column Family	Column Name	Value
1001	personal_info	name	John Doe

1001	personal_info	age	30
1001	personal_info	email	john@example.com
1001	address_info	street	123 Main St.
1001	address_info	city	Springfield
1001	address_info	zipcode	12345

HBase in ADBMS

In the context of **Advanced Database Management Systems (ADBMS)**, HBase plays a crucial role in **big data management** and **real-time analytics**. It is ideal for scenarios where:

1. High Scalability and Big Data:

- HBase is designed to scale to handle petabytes of data, making it suitable for big data applications, especially when the data is too large or too unstructured for traditional relational databases.

2. Low-Latency, Real-Time Operations:

- Unlike traditional databases, HBase is optimized for low-latency reads and writes, making it suitable for real-time analytics, web applications, and sensor data processing where you need quick access to large datasets.

3. Integration with Hadoop Ecosystem:

- HBase works seamlessly with other components of the **Hadoop ecosystem**, such as **MapReduce**, **Hive**, **Pig**, and **Spark**. It is often used as a storage layer for **Hadoop** for large-scale data processing and analytics.

4. Handling Semi-Structured and Unstructured Data:

- HBase is suitable for storing **semi-structured** data like logs, social media data, or sensor readings. Traditional relational databases struggle with such data because of their rigid schema requirements.

5. Real-Time Data Access for Web Applications:

- Many web applications require real-time data access for quick updates and queries. HBase allows applications to handle large amounts of real-time data with low latency.

Cassandra

Apache Cassandra is a powerful, highly scalable, and fault-tolerant **NoSQL** database that excels in managing large volumes of data across distributed systems. It is well-suited for applications that require high availability, fast writes, and horizontal scalability. As part of **Advanced Database Management Systems (ADBMS)**, Cassandra is particularly useful in **big data** applications, **real-time analytics**, and **distributed systems** that need to handle large datasets with minimal downtime.

Key Features of Apache Cassandra

1. Scalability:

- Cassandra is designed to scale horizontally by adding more nodes to the cluster. As the amount of data grows, new nodes can be added without downtime.
- Data is automatically distributed across nodes using a technique called **partitioning**, and there is no central master node, making it easy to scale the system.

2. High Availability:

- One of Cassandra's key design goals is high availability. Cassandra achieves this by replicating data across multiple nodes. If one node fails, data is still accessible from other nodes.
- It supports **multi-datacenter replication**, meaning data can be replicated across different geographic locations for disaster recovery and high availability.

3. Fault Tolerance:

- Cassandra is built to tolerate node failures. It uses **replication** to ensure that data is stored on multiple nodes. Even if one or more nodes fail, the system continues to function without data loss.

4. Decentralized Architecture:

- Cassandra uses a **peer-to-peer** architecture, where all nodes in the cluster are equal, and there is no single point of failure. Each node is responsible for managing a portion of the data, and the system remains available even if some nodes go down.

5. Flexible Schema:

- As a **NoSQL** database, Cassandra has a flexible schema. It allows for easy addition of new columns without downtime, making it ideal for applications with evolving data models.

6. Write-Optimized:

- Cassandra is optimized for high write throughput, making it suitable for write-heavy workloads. It supports **eventual consistency** by default, allowing for fast writes at the cost of immediate consistency across all replicas.

7. Tunable Consistency:

- Cassandra offers tunable consistency, which means you can adjust the level of consistency for different operations (e.g., **strong consistency**, **eventual consistency**, or **custom consistency levels**) based on your application's needs.

8. Column Family Data Model:

- Cassandra uses a **column-family** data model similar to Google's **Bigtable**. Data is organized into **tables**, with each table consisting of rows identified by a **primary key**. Each row can have an arbitrary number of columns, which can be added dynamically.

Cassandra Architecture

Cassandra's architecture is based on several key components that work together to ensure scalability, high availability, and fault tolerance:

1. Nodes:

- A Cassandra cluster consists of multiple nodes, each of which stores a subset of the data.
- Each node is independent, meaning there is no master node. All nodes are equally capable of handling both read and write requests.

2. Ring Architecture:

- Cassandra uses a **ring-based** architecture for data distribution. Each node is assigned a range of data based on its **token**. The data is distributed across the nodes in a **circular** manner.

- There is no central master, and each node is responsible for a specific portion of the data. If a node fails, the system can automatically reassign the data to other nodes.

3. Replication:

- Data is replicated across multiple nodes to ensure fault tolerance. Cassandra allows users to configure the number of replicas for each piece of data.
- **Replication Factor (RF)** determines how many copies of the data are maintained. For example, if RF = 3, there will be three copies of the data on different nodes.

4. Partitioning:

- Cassandra divides data into **partitions**. Each partition is responsible for a subset of the data, and the partition key determines where the data will be stored in the cluster.
- Partitioning ensures that data is distributed evenly across the cluster.

5. Consistency Levels:

- Cassandra provides tunable consistency levels for both read and write operations. You can configure how many replicas must acknowledge a request before it is considered successful.
 - **ONE**: Only one replica must acknowledge the operation.
 - **QUORUM**: A majority of replicas must acknowledge the operation.
 - **ALL**: All replicas must acknowledge the operation.
- This flexibility allows Cassandra to balance consistency, availability, and partition tolerance (CAP Theorem) based on application needs.

6. Gossip Protocol:

- Cassandra uses the **gossip protocol** for node-to-node communication. It helps nodes in the cluster discover and maintain information about each other's health and state.
- This protocol ensures that all nodes are aware of the cluster's topology and that data can be routed to the correct node.

7. Commit Log:

- Cassandra writes all data to a **commit log** to ensure durability. Once data is written to the commit log, it is then written to memory and eventually flushed to disk.
- The commit log provides durability, meaning that even in the event of a crash, Cassandra can recover data that was not yet written to disk.

Cassandra Data Model

Cassandra's data model is based on the **wide-column store** paradigm, which is a hybrid between **key-value** and **column-oriented** models.

- **Tables**: The primary structure for storing data in Cassandra is a **table**. A table consists of rows and columns, similar to relational databases.
- **Primary Key**: Each row is uniquely identified by a **primary key**. The primary key is composed of the **partition key** (determines the node where the row will be stored) and optional **clustering columns** (defines how the data is sorted within the partition).
- **Columns**: Each row can have a variable number of columns. Columns are grouped into **column families**, and columns can be added dynamically.

Example of Cassandra Table:

Consider a table for storing **user activity** data in a social media application:

```
CREATE TABLE user_activity (  
  user_id UUID,  
  activity_id UUID,  
  activity_time TIMESTAMP,  
  activity_type TEXT,  
  PRIMARY KEY (user_id, activity_time)  
);
```

- **Partition Key:** user_id (This determines which node in the cluster the data for a specific user will be stored on).
- **Clustering Key:** activity_time (This determines the order in which the rows are stored within a partition).

The table might store rows like this:

user_id	activity_time	activity_id	activity_type
12345	2025-03-19 10:00:00	67890	Post
12345	2025-03-19 11:00:00	67891	Like
67890	2025-03-19 10:30:00	98765	Comment

Here, user_id is the partition key, and activity_time is the clustering key. This allows for efficient queries by user and activity time.

Cassandra in ADBMS

In the context of **Advanced Database Management Systems (ADBMS)**, Cassandra serves as a key player in managing large-scale, distributed systems that require high availability, fault tolerance, and performance at scale. Cassandra is suitable for:

1. Big Data Applications:

- Cassandra can handle massive amounts of data, making it suitable for **big data** applications like log management, sensor data, and clickstream analysis.

2. Real-Time Data:

- For real-time applications like **social media analytics**, **user activity tracking**, and **IoT data storage**, Cassandra offers low-latency writes and can handle large volumes of data efficiently.

3. Multi-Datcenter Deployment:

- Cassandra's ability to support **multi-datcenter replication** is useful in applications requiring **disaster recovery**, **geographical distribution**, and **global applications**.

4. Write-Heavy Workloads:

- Applications that involve **write-heavy workloads**, such as recommendation engines, logging systems, and time-series data, benefit from Cassandra's write-optimized architecture.

CAP Theorem

The **CAP Theorem** (also known as **Brewer's Theorem**, after its creator, Eric Brewer) is a fundamental concept in distributed database systems and plays a critical role in the design and functioning of modern **Advanced Database Management Systems (ADBMS)**, especially in the context of **NoSQL databases** like **Cassandra**, **MongoDB**, **Couchbase**, etc.

The CAP Theorem states that a distributed database system can provide at most **two** of the following **three** guarantees:

1. **Consistency (C)**
2. **Availability (A)**
3. **Partition Tolerance (P)**

A database system can choose to sacrifice one of these properties to ensure the other two, but it cannot guarantee all three simultaneously in the presence of network partitions or failures.

Explanation of the Three Properties in CAP Theorem

1. Consistency (C):

- **Consistency** means that every read operation will return the most recent write (or an error) for a given piece of data, ensuring that all nodes in the system have the same data at any given time.
- In other words, after a write operation, any subsequent read will always return the updated data across all nodes in the system.
- **Example:** If two users try to update the same record at the same time, one of the updates will be reflected in all subsequent reads, ensuring that everyone sees the same data at the same time.

2. Availability (A):

- **Availability** means that the system will always be available to respond to a read or write request, even if some nodes are down or unreachable.
- In other words, every request (whether it is a read or write operation) will receive a response, even if some of the nodes in the system are not functioning.
- **Example:** If a server goes down in a distributed system, the system can still respond to the client's requests using data from another node, ensuring that users can still access and update data.

3. Partition Tolerance (P):

- **Partition Tolerance** means that the system will continue to function even if network partitions occur between nodes, where some nodes cannot communicate with others.
- A **partition** is when a network failure or fault causes parts of the system to be isolated from one another. Partition tolerance ensures that even if some nodes or

parts of the network are temporarily inaccessible, the database will still function normally (with the trade-off of possibly not providing consistency).

- **Example:** If two nodes in the system cannot communicate due to a network partition, the system can still operate, with the risk that the data on these nodes might temporarily be inconsistent.

Understanding the CAP Theorem with Examples

Let's explore some examples to better understand how the CAP Theorem plays out in real-world systems.

1. Cassandra (AP System - Availability and Partition Tolerance)

- **Cassandra** is an example of a system that prioritizes **Availability** and **Partition Tolerance** over **Consistency**.
- Cassandra can handle network partitions gracefully by allowing updates to occur even when nodes are partitioned. However, during this time, the system may return different versions of data to different clients, leading to potential **eventual consistency** (i.e., after a period of time, all nodes will converge to the same data state).
- **Example:** Suppose you have a distributed Cassandra cluster with three nodes. If one node is unreachable due to a network partition, the remaining two nodes can still accept writes and serve reads. However, when the partition is resolved, Cassandra will eventually synchronize the data across the nodes to ensure consistency.

2. HBase (CP System - Consistency and Partition Tolerance)

- **HBase**, when deployed with **HDFS** (Hadoop Distributed File System), typically prioritizes **Consistency** and **Partition Tolerance**.
- In this case, when a partition occurs between nodes, HBase may reject or block writes to ensure that consistency is maintained across the system. While this may lead to reduced availability during network partitions, the system will maintain a consistent state of the data.
- **Example:** Consider an HBase system with multiple region servers. If a partition occurs, HBase will not allow writes to occur on a region that has become unreachable to maintain consistency across all copies of the data. Once the partition is resolved, HBase will synchronize the data to ensure that all nodes have consistent copies.

3. MongoDB (AP System - Availability and Partition Tolerance)

- **MongoDB**, in its default configuration, prioritizes **Availability** and **Partition Tolerance**.
- MongoDB is a document-based database that allows data to be replicated across multiple nodes. During network partitions, MongoDB will allow reads and writes to continue even if some of the nodes are not reachable, sacrificing consistency. The system relies on **eventual consistency** to resolve data discrepancies once the partition is resolved.
- **Example:** In a MongoDB replica set, if one node goes down or is unreachable due to a network partition, the other nodes can still accept writes and reads. However, different

nodes might have different versions of the same document until the partition is resolved, at which point MongoDB will synchronize the data across the nodes.